

Forecasting Fluctuations in the Financial Index Using a Recurrent Neural Network Based on Price Features

Yu-Fei Lin, Tzu-Ming Huang, Wei-Ho Chung , and Yeong-Luh Ueng , *Senior Member, IEEE*

Abstract—Profits can be made from a trading strategy where long or short positions are placed in advance, based on the ability to forecast a future stock price or index, such as the closing or opening price. In addition to predicting stock index values, a prediction of the sign for the difference between closing and opening prices is important in order to earn a profit. This article presents an approach based on a Recurrent Neural Network (RNN) to forecast the opening price, the closing price, and the difference between them. Compared to previously reported approaches that were based on machine learning, the method proposed here emphasizes the pre-processing of the data, including the normalized first order difference method, as well as the focusing on the characteristics of the stock data, such as the zero-crossing rate (ZCR), which denotes the ratio of changes in the sign within a specific time interval. We propose a decision-making approach that is based on an estimate of both the ZCR and the cross-validation data so as to enhance the ability to forecast the difference between the opening and closing prices. We apply our technique to the S&P500 (Standard & Poor's 500) and the Dow Jones stock indices. As indicated by the results, our method can achieve a better performance compared to previous work.

Index Terms—Deep learning, Stock price prediction, Recurrent neural network (RNN).

I. INTRODUCTION

IN RECENT YEARS, the emergence of research works in financial time sequence prediction has become an important topic. One of the more challenging tasks, however, is predicting the trends of a financial time series, such as the index, currency or stock price. Amongst the reasons for this are the numerous factors that might impact the trend, including the local economy, the political situation, the loss and profit for an individual company,

or the exchange rate, etc. In order to make a profit on the stock market by placing long or short positions, forecasting the stock index value, such as the opening and closing price, is crucial. However, not only knowing the value of the stock index, but also the sign of its difference can enable investors to earn profits more easily. If an investor can correctly predict whether the direction of the difference between the opening and closing prices will be up or down, by buying or selling assets at different prices, the investor can potentially earn a profit. There are many price-range based strategies that can be used to earn a profit [1]–[3]. An investor may make profits more easily using appropriate trading strategies [4], [5]. The study presented in [6] presented a fuzzy approach that could be used to predict the TAIEX index. The authors of [7] integrated wavelet analysis and TSK fuzzy rules in an approach to forecasting the stock price. The proposal outlined in [8] presented a technique that was based on Genetic Fuzzy Systems that was used to forecast the IBM stock index. This earlier work obtained effective outcomes that decreased regression errors, including the root mean square error (RMSE) and the mean absolute error (MAE), since fuzzy approaches that follow the **minmax** rule relating AND and OR operations are able to simulate a variety of behaviors. The investigations explained in [9] and [10] implement the autoregressive integrated moving average (ARIMA) model to enable time series predictions. The authors of [11] presented a technique they termed the cross correlation ARIMA (CARIMA) that could be used to forecast stock prices on the SET50. Since ARIMA assumes that the relationships between both dependent and independent variables are linear, it tends to be unstable when predicting a non-linear financial time series. Also, ARIMA assumes that the standard deviation for the errors in the model is constant [12].

Since we are able to resolve many non-linear prediction problems using machine learning, studies that employ this technique to forecast a financial time series are in the majority. A support vector machine (SVM)-based approach was investigated by the authors of [13] and [14] for the financial time series forecasting. The conversation about whether or not the model will be subject to overfitting, however, greatly depends on the kernel, as it can be somewhat sensitive to the data [15]. The writers of [16] showed that an Artificial Neural Network (ANN) is able to achieve regression errors lower than those achieved using ARIMA. Further, the authors in [17] applied a deep neural network (DNN) to predict the mid-price using feature engineering. The authors of [18] combined both reinforcement learning and deep learning for financial trading. Some research has proposed the frequency component as a feature to enhance the performance of

Manuscript received June 27, 2019; revised October 28, 2019 and December 19, 2019; accepted January 11, 2020. Date of publication February 19, 2020; date of current version September 23, 2021. This work was supported in part by the Ministry of Science and Technology under Grant MOST 107-2628-E-007-005-MY3 and Grant MOST 107-2628-E-007-005-MY3 and in part by the projects in Research Center for Information Technology Innovation, Academia Sinica. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Yun Li. (*Corresponding author: Yeong-Luh Ueng.*)

Y.-F. Lin is with the Department of Electrical Engineering, National Tsing Hua University, Hsinchu 30013, Taiwan (e-mail: yuifeilin@gmail.com).

T.-M. Huang is with the Institute of Communications Engineering, National Tsing Hua University, Hsinchu 30013, Taiwan (e-mail: sl06064527@ml06.nthu.edu.tw).

W.-H. Chung is with the Institute of Communications Engineering, National Tsing Hua University, Hsinchu 30013, Taiwan (e-mail: whchung@ee.nthu.edu.tw).

Y.-L. Ueng is with the Department of Electrical Engineering and the Institute of Communications Engineering, National Tsing Hua University, Hsinchu 30013, Taiwan (e-mail: ylueng@ee.nthu.edu.tw).

Digital Object Identifier 10.1109/TETCI.2020.2971218

stock price prediction based on machine learning. The authors of [19] extracted the cycle factors and implemented an attention mechanism to train the DNN.

The method proposed in [20] was based on time series reconstruction and a convolution neural network (CNN) to predict the stock trends. The authors of [21] analysed the relationship between a group of companies with CNN to predict the stock price. Great success has been achieved in recent years via the use of Recurrent Neural Network (RNN), particularly in the speech area [22], [23]. In addition to basic RNN, another common RNN structure, known as long short-term memory (LSTM), can be used to effectively identify valuable characteristics from the previous cycles. In [24], the proposal presented employed an LSTM approach to forecast the closing price of a commodity, since the LSTM uses huge networks that are able to simulate various complex behaviors and, consequently, fit the data successfully. The study presented in [25] proposed using discrete Fourier transform (DFT) to decompose trading patterns into various frequency components, and applied an LSTM unit to predict stock price. Natural language processing (NLP) has recently become increasingly popular. The authors of [26], [27] [28] enhanced the accuracy of stock price prediction with details from financial news sources or other reports.

Any difference in price should also be considered in addition to the stock index value, such as the opening price and the closing price, so as to accommodate practical applications. However, the majority of earlier work tends to ignore this objective. Similarly, when we employ the RMSE or MAE techniques, etc. to evaluate the model, it is not sufficient to earn a profit from the stock market. Even though the studies reported above were able to achieve great results through the selection of different machine learning structures, they tended to ignore the data characteristics. By being able to take advantage of these characteristics from data, we can improve our prediction model.

In this study, not only do we focus on the machine learning structure, but we also investigate the pre-processing and characteristics of the data, such as the normalized first order difference method and the Zero-Crossing Rate (ZCR), which is the ratio of the total number of changes in sign in two consecutive time instants within a given time interval. We then use ZCR to first analyze the data, and then propose an approach that may be used to forecast the price range based on the outcome of the ZCR.

Performing the first order difference is used to eliminate the trend in the data so that we can focus on the volatility of the data. We use normalization to prevent the output from becoming affected by the imbalanced training input too greatly. We then finally evaluate our proposed approach based on MAE, RMSE, and the hit rate for the predictions. From the results of our research, it can be seen that an accuracy of around 58% is achieved when considering the stock price range, and, when compared to the SVM and LSTM based methods, the same regression error is achieved. In addition, we have compared the existing research with our research in Table I.

The remainder of this paper is organized as follows. We will briefly explain the structure of the basic RNNs and the LSTM in Section II. In Section III, a methodology is proposed that is implemented to forecast the opening and closing prices, as

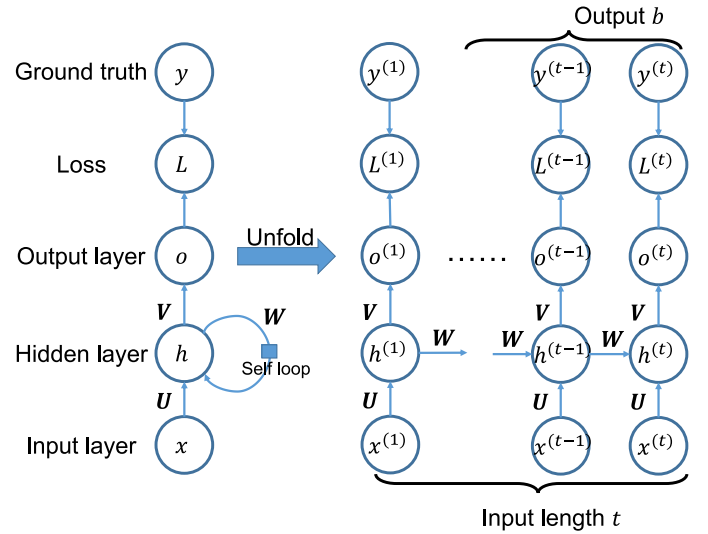


Fig. 1. Unfolded RNNs.

well as the price range value. We then propose a methodology in Section IV that explains how we use ZCR to predict either positive or negative values for the price range. Finally, a brief summary is provided in Section V.

II. PRELIMINARY

A. RNNs [29]

Except the feedback loop, the graph on the left side of Fig. 1 illustrates a basic ANN, where an activation only flows in a single way. Since any neural network contains a ‘self-loop’ state that includes a weight, it can be represented with regard to a time axis. From the right-hand side of Fig. 1, for a time index $i \leq t$, it can be seen that the recurrent neuron $\mathbf{h}^{(i)} \in \mathbb{R}^{u \times 1}$ and output $\mathbf{o}^{(i)} \in \mathbb{R}^{l \times 1}$ can be written as:

$$\mathbf{h}^{(i)} = \sigma(\mathbf{U}\mathbf{x}^{(i)} + \mathbf{W}\mathbf{h}^{(i-1)} + \mathbf{c}_1^{(i)}), \quad (1)$$

$$\mathbf{o}^{(i)} = \mathbf{V}\mathbf{h}^{(i)} + \mathbf{c}_2^{(i)}, \quad (2)$$

where t is the input length and $\sigma()$ is the activation function. Therefore, there are u neurons in each layer and a total of l labels in $\mathbf{o}^{(i)}$. Write input $\mathbf{x}^{(i)} = [f_1^{(i)}, \dots, f_q^{(i)}]^T$, where $f_k^{(i)}$ denotes the k -th feature at time i . $\mathbf{c}_1^{(i)} \in \mathbb{R}^{u \times 1}$ and $\mathbf{c}_2^{(i)} \in \mathbb{R}^{l \times 1}$ denote the bias term for the hidden and output layers, respectively. There are three weighting matrices, i.e., $\mathbf{U} \in \mathbb{R}^{u \times q}$, $\mathbf{W} \in \mathbb{R}^{u \times u}$, and $\mathbf{V} \in \mathbb{R}^{l \times u}$.

The purpose of using the activation function is to ensure that the model is a non-linear machine. Common activation function choices are sigmoid, tanh, and ReLU functions as shown in Fig. 2. Although the ReLU activation function is commonly used, we use sigmoid and tanh as our activation functions for the reasons that will be described in the next subsection.

B. LSTM [30]

LSTM [30], in particular, is an advanced structure for an RNN, enabling the RNN to predict a time series more accurately.

TABLE I
COMPARISONS OF THE INDEX PREDICTION METHODS

Ref.	Data source	Preprocessing method	Algorithm	Performance
[6]	TAIEX	Fuzzify historical data	Fuzzy time series and variation groups	RMSE: 91.98
[7]	Taiwan Stock Exchange	Haar- wavelet transform	TSK-Fuzzy	MAPE: 0.792
[8]	IBM	Stepwise regression	GFS	MAPE: 0.65
[9]	NYSE, NSE	First order difference	ARIMA	-
[10]	Sensex, Nifty	First order difference	ARIMA	-
[11]	SET50	Unit Root Test	CARIMA	MAE: 2.85
[13]	Chicago Mercantile Market	-	ASVM	NMSE: 1.005 MAE: 0.8033 DS: 48.54
[14]	Taiwan Stock Market	Feature selection	SVM	Acc: 68.18%
[17]	Nordic Stock Market	Statistical filtering, Feature extraction	DNN	F1 score: 53% RMSE: 87.7
[18]	Euro-Dollar exchange rates	Reward shaping	DNN, Reinforcement learning	-
[19]	US Stock Market	Weekly cycle embedding tagging	CEAM	MAE: 66.453 MAPE: 8.152 RMSE: 13.754
[20]	S&P500	Time series reconstruction	CNN	Recall: 74.753% precision: 55.44% F1 score: 63.67% Acc: 56.14%
[21]	CSI300	Construction of corporation network	LSTM, GCN	Acc: 57.98%
[24]	S&P500, NASDAQ	Data slicing	LSTM	MAE: 14.77 MAPE: 72.4% RMSE: 20.47 AMAPE: 71.45%
[25]	50 Stocks	Normalization	SFM, LSTM	MSE: 5.57%
[26]	CSI300, S&P500, financial news	Tokenized, Normalized, Aligned and Labeled	NBA	MSE: 0.147 Acc: 58.1%
[27]	S&P500, financial news	Relevant keyword extraction	Text-based DNN	Acc: 55.9%
[28]	Hang Seng Index, financial news	Sentimental mapping	Transfer learning, SVM	Acc: 56.8%
[34]	SET50	-	ANN, SVM	F-measure: 61.69% Acc: 56.3%
Ours	S&P500, Dow Jones	First order difference, Normalization	LSTM, ZCR	RMSE: 12.933 MAE: 10.44 Acc: 59.4%

The biggest difference, as shown in Fig. 3, is that more control gates are contained in the basic RNN structure. The tanh function is chosen as the activation function for the input while the sigmoid function is used for the input, output, and forget gates. Since the sigmoid function outputs a value between 0 and 1, this function is able to control a suitable portion of information throughout the gates. In contrast, in order to overcome issues related to vanishing gradients, we require a function whose second order derivative can be sustained for an extended range before it returns to zero, which can be achieved by the tanh function. Let the output values for the output gate, the input gate, and the forget gate at time i be denoted by

$g_o^{(i)}$, $g_x^{(i)}$, and $g_f^{(i)}$, respectively. State $s^{(i)}$ is updated as below:

$$s^{(i)} = g_f^{(i)} s^{(i-1)} + g_x^{(i)} x_{in}^{(i)}, \quad (3)$$

where $x_{in}^{(i)} = \tanh(\mathbf{U}_{in} \mathbf{x}^{(i)} + \mathbf{c}_{in})$, $\mathbf{c}_{in}$ and $\mathbf{U}_{in}$ are, respectively, the bias and the weight matrix for $x_{in}^{(i)}$. Then, we have:

$$h^{(i)} = g_o^{(i)} \tanh(s^{(i)}). \quad (4)$$

By utilizing LSTM, we improve the long-term dependency problem, since LSTM can obtain more information from previous when compared to a basic RNN model.

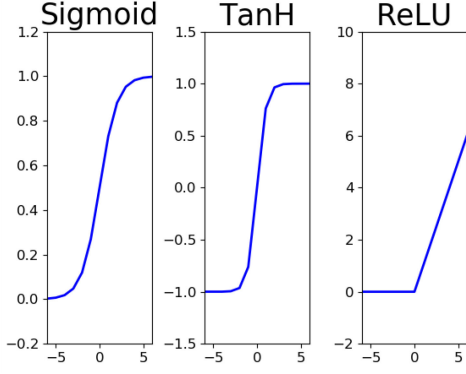


Fig. 2. Activation functions.

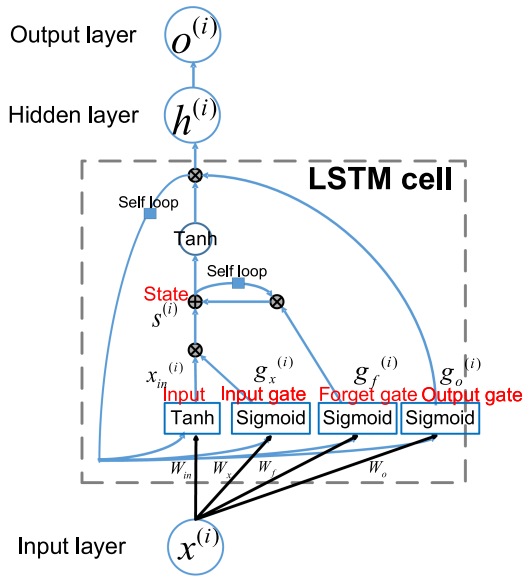


Fig. 3. An LSTM cell [30].

C. Creating and Optimizing Samples for the RNN Structure

Here, we assume that $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(L)}$ is sequence data, where L denotes the data length. The number of labels in output l is set to 1. X_j is the j -th input sample, while Y_j is the ground truth:

$$X_j = \{\mathbf{x}^{(j)}, \mathbf{x}^{(j+1)}, \dots, \mathbf{x}^{(j+t-1)}\}, \quad (5)$$

$$Y_j = \{f_k^{(j+t)}, \dots, f_k^{(j+t+b-1)}\}, \quad (6)$$

where $1 \leq j \leq L - b - t + 1$ and k is feature index of the target, as shown in Fig. 4.

Each of the samples is divided into the training part, the cross-validation part, and the testing part. Using the training and cross-validation data, the model can be optimized. Then, the model can be evaluated using the testing data. As shown in Fig. 1, once both the ground truth and the output are obtained, we are able to calculate the Loss as part of the optimization process. In order to ensure that the Loss is as small as possible, we need to obtain the optimal weights, which can be achieved through

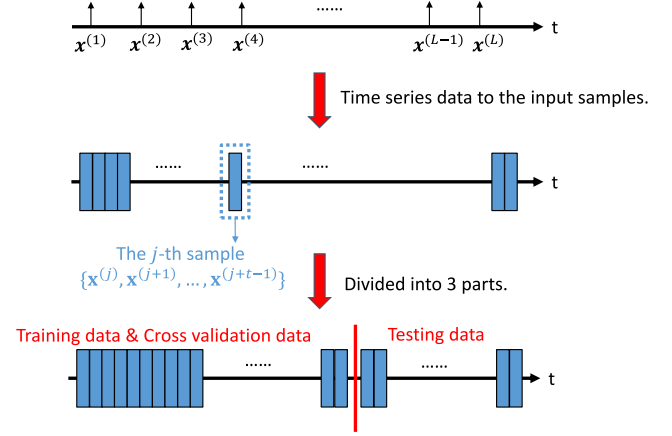


Fig. 4. Create samples for the RNN inputs.

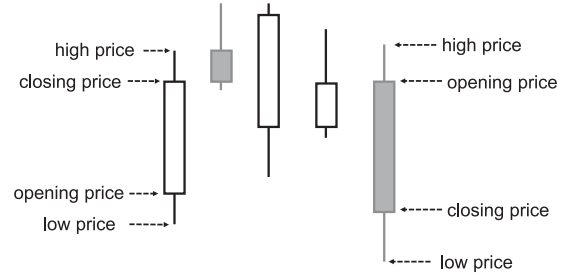


Fig. 5. Candlestick chart.

the use of the least absolute shrinkage and selection operator (LASSO) [31] and the stochastic gradient descent algorithm (SGD) [32].

III. FORECAST THE OPENING, THE CLOSING AND THE DCO VALUE

Fig. 5 shows the candlestick chart of stock market for the daily stock price information. We propose a method in this section that can be employed to forecast the opening price, the closing price, and the DCO, respectively denoted as o , c and r , where the term DCO is the difference between the closing price and the opening price. Fig. 6 shows an associated prediction flow chart. By substituting the training data and the ground truth, the output of the model can be regarded as the projected value with respect to different targets, i.e., o , c or r . In other words, we have three different models that are trained to respectively estimate the o , c and r values.

The pre-processing stage of the stock value model involves two steps [35]. First, a first order difference evaluation is performed on the original stock data in order to eliminate the trend in the original data. Subsequently, we normalize the data. The intent of data normalization is to ensure that the difference between the data would not be too significant.

A. Stock Index Value Prediction

In this section, the stock index value prediction using the normalized first order difference data is presented. Suppose

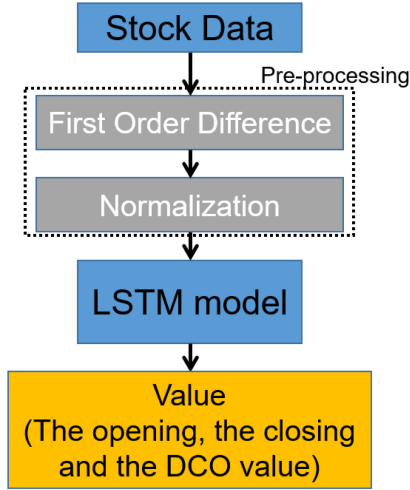


Fig. 6. Flowchart for predicting the opening and closing prices, and the DCO value.

$\mathbf{v}^{(1)}, \dots, \mathbf{v}^{(L)}$ represents a sequence of stock data for L trading days. There are seven features involved, including o , c , r , the lowest daily price, the highest daily price, the Volume, and the DHL, where the DHL represents the difference between the highest and the lowest daily prices. Hence, $\mathbf{v}^{(i)}$ is a 7×1 vector.

The output length b is set to be equal to 1, and the target feature will be set to the value of o , c and r , depending on the different models, which means that we are using information from past t days to predict these values for the next day.

Let us first talk about the opening price model. The closing price and the DCO can also be obtained by using a similar approach. We normalize the first order difference in the stock data during the training stage such that the model can be more easily trained in a more stable manner. The goal of normalization is to change the values of the numeric columns in the dataset to a common scale, without distorting any differences in the ranges of the values. The data for the first order difference can be written as $\mathbf{v}^{(i)} - \mathbf{v}^{(i-1)}$. The j -th input sample for the Opening price model is $\Delta_j = \{\mathbf{d}^{(j)}, \dots, \mathbf{d}^{(j+t-1)}\}$, and $o^{(j+t)} - o^{(j+t-1)}$ is the corresponding ground truth Y_j , where $\mathbf{d}^{(i)}$ is the result of $\mathbf{v}^{(i)} - \mathbf{v}^{(i-1)}$ after normalization for $2 \leq j \leq L - t$ and $j \leq i \leq j + t - 1$.

After the model is trained using Δ_j and Y_j , we can obtain a predicted output, denoted as $\hat{Y}_j$, as shown in Fig. 7. In this case, however, we actually want to predict the value of $o^{(j+t)}$ instead of $\hat{Y}_j$. An estimate of $o^{(j+t)}$, $\hat{o}^{(j+t)}$, can be calculated by using

$$\hat{o}^{(j+t)} = o^{(j+t-1)} + \hat{Y}_j. \quad (7)$$

B. Model Settings and Performance Metrics

Actually, both the training model and the results will be greatly influenced by using different data segments. As a result, the original stock data is first divided into a number of segments, and then we train the model and evaluate each segment individually, as shown in Fig. 8. Finally, all the results are averaged. We collected the stock index information for the Dow Jones and the

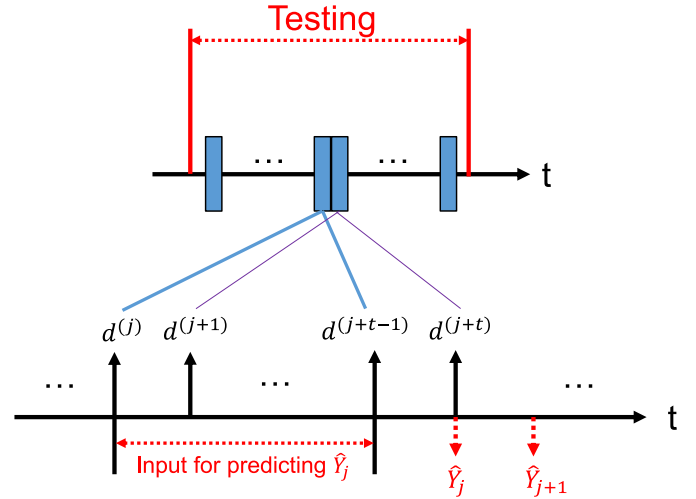


Fig. 7. The relationship between the data for the stock index prediction.

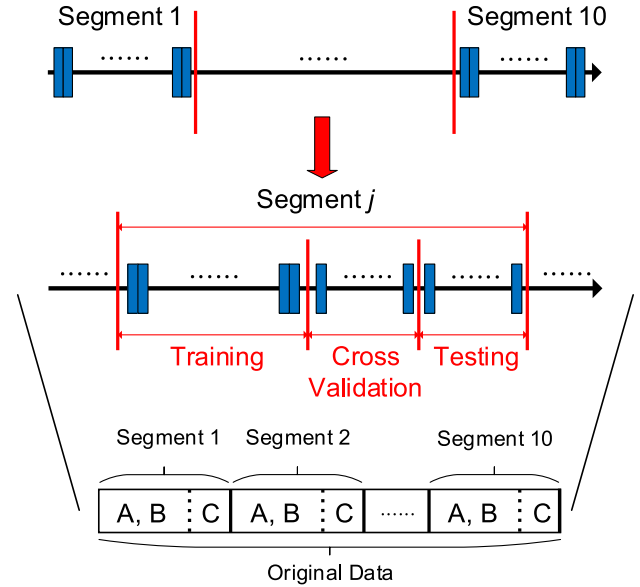


Fig. 8. Dividing the data into segments.

S&P500 indices from the Yahoo! Finance website, where stock index data is available for the Dow Jones and the S&P500 for a total of 5540 days from 2/1/2005 to 12/30/2016. We divided the data into 10 segments equally and each segment contains a training, a cross-validation, and a testing set, referred to as Set A, B and C, respectively. The cross-validation samples in set B are used to avoid overfitting. There are two sizes for the testing samples, 20 and 15.

The RNN consists of three layers and each layer contains 100 neurons. When optimizing the model, we first initialize the weight with He initializer [33], then optimize the weights by the SGD algorithm [32]. The learning rate is 0.001. With respect to the estimated opening price ($\hat{o}^{(i)}$), the closing price ($\hat{c}^{(i)}$), and the DCO value ($\hat{r}^{(i)}$) from (7), the relevant regression errors,

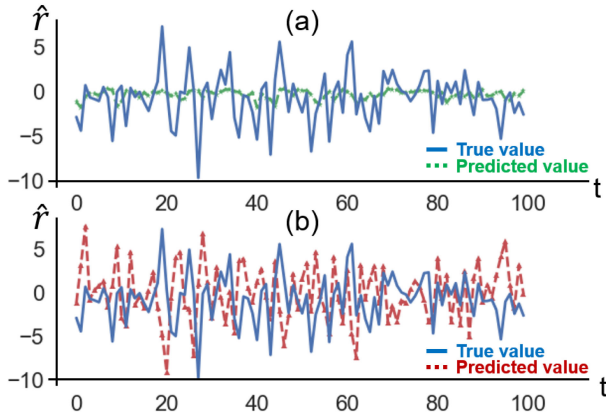


Fig. 9. (a) Predictions based on the original stock data; (b) Predictions based on the first order difference data.

i.e., RMSE and MAE values, are obtained according to:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2}, \quad (8)$$

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}^{(i)} - y^{(i)}|, \quad (9)$$

where $\hat{y}^{(i)}$ is the estimated value of $y^{(i)}$.

In addition to the regression errors, our metrics of interest include the hit rate with respect to the up or down movement of the DCO, i.e., the Prediction of the Sign (POS) is used to ensure the accuracy of our model, and is defined as:

$$\text{POS} = \frac{\sum_{i=1}^N |\text{sgn}(r^{(i)}) + \hat{s}^{(i)}|/2}{N}, \quad (10)$$

where $\text{sgn}()$ follows:

$$\text{sgn}(x) = \begin{cases} 1, & \text{if } x > 0 \\ -1, & \text{if } x < 0 \end{cases}, \quad \text{where } x \in \mathbb{R}, \quad (11)$$

and $\hat{s}^{(i)}$ is the estimated $\text{sgn}(r^{(i)})$.

C. Simulation Results

We now use the results from the simulations to illustrate the advantages of the proposed pre-processing methods. Fig. 9 shows data for both the true and the estimated DCO within a time interval. Fig. 9(a) shows the true and the estimated DCO data that were predicted by the model trained without using the first order difference data. It can be observed that the estimated DCO data does not accurately follow the true data, and oscillates near the zero axis. Fig. 9(b) illustrates both the true and the estimated DCO data that was predicted by the model trained employing data from the first order difference, and distinctly fits the data better, or, at least, indicates a relative similarity in the orders of magnitude for the data. The following is a discussion of our approach to normalization. There are actually two reasons that we perform normalization. The first is to avoid excessive

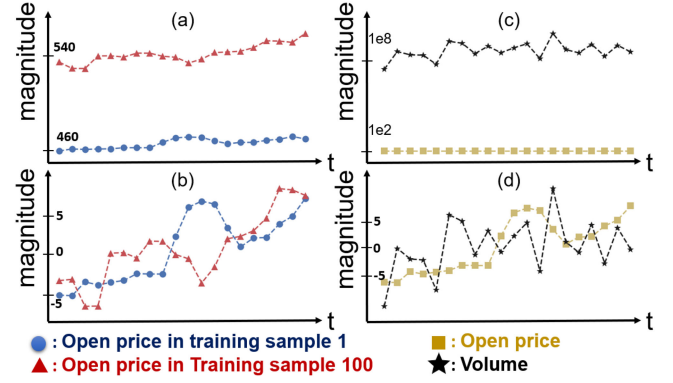


Fig. 10. Illustration of the normalization process.

differences for the same feature at different times. Fig. 10(a) shows that the data for the opening price is very different between two different input samples. It can be seen from Fig. 10(b) that the disparity between the two data sets is diminished once the data is normalized. The second reason is so that we can scale different features into a relatively similar range. We notice from Fig. 10(c) that the volume data is much larger than other data, such as opening price data. If a huge disparity exists between two features, it will cause significant difficulty during the training. Fig. 10(d) shows the data once it has been normalized.

Table II provides a comparison of the SVM- and LSTM-based methods without using the proposed pre-processing and the proposed method, where the results are denoted as SVM, LSTM, and M, respectively. The table shows the results for the regression errors when the opening price, closing price, and DCO for the S&P500 and Dow Jones are predicted using SVM, LSTM, and M, where subscripts 15 and 20 represent the evaluation of the method where $N = 15$ and $N = 20$, respectively. The proposed method is able to achieve fewer regression errors for the opening price and the closing price than other methods. However, SVM and LSTM can achieve better results for the DCO. In addition to the regression errors, Table III shows the resultant hit rate (POS) for the DCO, where we let the estimated sign for the DCO value be equal to $\text{sgn}(\hat{r}^{(i)})$. In this case, all the results are only close to 50%.

Fig. 11 illustrates why applying the first-order difference is effective for reducing the regression errors in the opening price and closing price. Conversely, it does not help much for the DCO. Fig. 11(a) shows the opening price for the Dow Jones index. Before applying the first-order difference, the magnitude of the data is large and unstable. After applying the first-order difference, the data magnitude is reduced and the bias term is close to zero, so it is easier for the machine to learn the data. However, the DCO for the Dow Jones shown in Fig. 11(b) is small in magnitude and acts in a more stable manner. Since the DCO provides the difference between the closing and opening prices, it has been differentiated once. This means that the first-order difference will no longer help the DCO data to become more stable. However, in order to describe our model uniformly, we still include the first-order difference in our flow chart. The first-order difference for the DCO can be removed without affecting the prediction performance.

TABLE II
COMPARISONS OF THE S&P500 AND THE DOW JONES FOR THE AVERAGE RMSE AND MAE

SUBJECT		SVM ₂₀		LSTM ₂₀		M ₂₀		SVM ₁₅		LSTM ₁₅		M ₁₅	
		RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
SP'	Open	36.6	29.0	66.5	61.9	5.3	4.0	36.2	29.0	59.0	56.0	5.718	4.46
	Close	44.5	36.6	62.1	62.5	14.1	14.1	43.7	36.6	54.3	48.3	13.3	10.4
	DCO	13.2	10.5	13.1	10.6	15.3	12.5	13.3	10.8	13.4	11.0	15.7	12.7
DJ'	Open	661.9	604.5	657.0	616.4	44.6	34.1	656.8	600.0	612.6	577.4	44.9	35.0
	Close	704.3	652.4	543.4	508.9	112.3	86.9	696.9	648.7	579.2	544.7	114.3	89.6
	DCO	112.2	88.6	113.6	88.6	120.2	96.2	112.3	88.7	112.2	87.9	122.0	100.6

TABLE III
COMPARISON OF THE AVERAGE POS BETWEEN THE PROPOSED METHOD AND PREVIOUSLY REPORTED METHODS FOR THE DCO HIT RATE FROM THE S&P500 AND THE DOW JONES

SUBJECT	SVM ₂₀	LSTM ₂₀	M ₂₀	SVM ₁₅	LSTM ₁₅	M ₁₅
SP' DCO	0.505	0.523	0.455	0.447	0.453	0.5
DJ' DCO	0.48	0.485	0.525	0.433	0.445	0.504

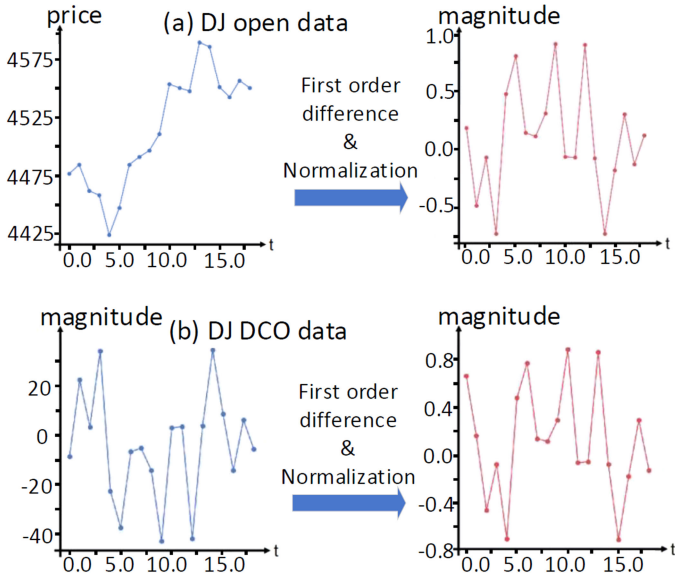


Fig. 11. Pre-processing on the opening price and the DCO data.

To predict the DCO value, as well as training the model where r is set as the ground truth, the estimated $r^{(i)}$ value can also be obtained from the output of both the closing price model and the opening price model by following $\hat{c}^{(i)} - \hat{o}^{(i)}$. Suppose that the combined model for predicting $\hat{c}^{(i)} - \hat{o}^{(i)}$ is denoted as H . Tables IV and V show the regression errors and the POS with regard to $\hat{c}^{(i)} - \hat{o}^{(i)}$, respectively. For both the S&P500 and the Dow Jones, the regression errors are similar to model M. Considering the hit rate, for the S&P500, it is not feasible to directly allow the estimated $\text{sgn}(r^{(i)})$ be equal to $\text{sgn}(\hat{c}^{(i)} - \hat{o}^{(i)})$ directly. However, model H can perform better to the Dow Jones whose result almost achieves 57% accuracy.

The results for both the POS and the regression errors are now used in order to illustrate the disadvantages when evaluating

the model employing only the results for the regression errors. Figs. 12(a) and 12(b) depict the DCO values when both the LSTM and the proposed methods, respectively, are used, where the regression error for both methods is similar. The proposed method, however, is much better than the LSTM in its ability to capture both the trend of the stock price and the sign of the DCO. Although the SVM, LSTM and the proposed approach share almost the same regression error values for the DCO, as shown in Table II, this does not mean that they possess the same ability to forecast the sign of the DCO. Consequently, in addition to being concerned with the regression errors, we also care about the POS.

In addition to the above, so that we can improve the ability to predict the DCO value, which can be written as $\hat{r}^{(i)} = |\hat{r}^{(i)}| \times \hat{s}^{(i)}$, we introduce a method that can be used to forecast the sign of the DCO value more precisely. Since we now have an improved estimate of the value for the DCO sign, we can further reduce the regression errors.

IV. UP OR DOWN FORECAST FOR THE DCO

In addition to the RNN model used to predict the DCO value, we propose a method to determine the DCO sign. If we can accurately forecast the sign of the DCO, it is easier to earn profits by placing either a long or short position in the stock market. The workflow for our proposed sign prediction of DCO is shown in Fig. 13. We are interested in two classification situations in this study, $\text{DCO} > 0$ and $\text{DCO} < 0$. We use the classification output $\hat{\rho}$, which is the estimate of the trend of the DCO, to replace the estimated value of r . The ZCR value, i.e., $\hat{z}$, is obtained using the ZCR model. In the decision-making phase, a combination of $\hat{\rho}$ and $\hat{z}$ is used to determine the value of the output $\hat{s}$, which is an estimate of the sign of r . Finally, the accuracy of $\hat{s}$ will be evaluated.

A. Decision-Making Process for the DCO Sign

In this section, we will mostly focus on the sign for the DCO value, i.e., $s^{(i)} = \text{sgn}(r^{(i)})$. However, the resulting outcome will not be accurate sufficiently. The following will show an approach that allows us to get a better estimate for $s^{(i)}$.

If $\rho^{(i)}$ is defined as:

$$\rho^{(i)} = \text{sgn} \left(r^{(i)} - r^{(i-1)} \right), \quad (12)$$

TABLE IV
CALCULATING THE REGRESSION ERRORS USING THE OPENING AND CLOSING MODELS FOR THE DCO VALUE FROM THE S&P500 AND THE DOW JONES

SUBJECT	H ₂₀		H ₁₅		M ₂₀		M ₁₅	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
SP' DCO	15.005	11.906	14.138	11.372	15.3	12.5	15.7	12.7
DJ' DCO	123.196	96.446	125.678	98.582	120.2	96.2	122.0	100.6

TABLE V
CALCULATING THE POS USING THE OPENING AND CLOSING MODELS FOR THE DCO VALUE FROM THE S&P500 AND THE DOW JONES

SUBJECT	H ₂₀	H ₁₅	M ₂₀	M ₁₅
SP' DCO	0.455	0.511	0.455	0.5
DJ' DCO	0.57	0.568	0.525	0.504

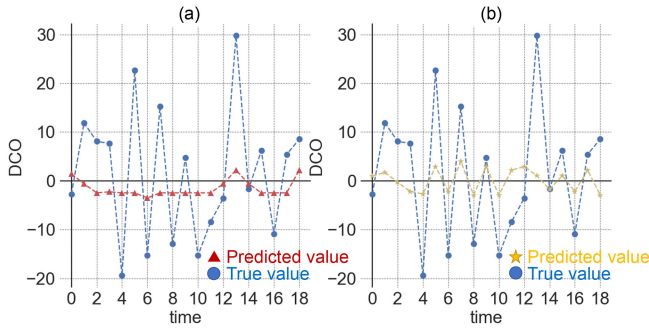


Fig. 12. Predicted DCO value using two different methods. (a) the LSTM method; (b) the proposed method.

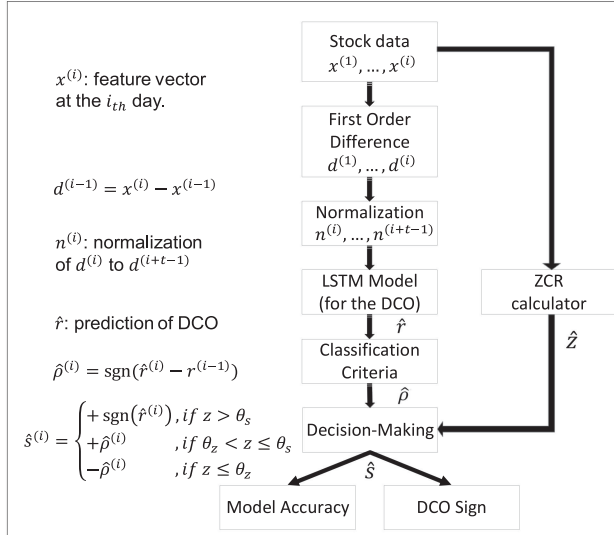


Fig. 13. Workflow for the proposed sign prediction of DCO.

$\rho^{(i)}$ can be viewed as a DCO trend. Since this trend is more regular, it is easier to make the price spread model learn the trend. An estimate of $\rho^{(i)}$, denoted as $\hat{\rho}^{(i)}$, can then simply follow:

$$\hat{\rho}^{(i)} = \text{sgn}(\hat{r}^{(i)} - r^{(i-1)}), \quad (13)$$

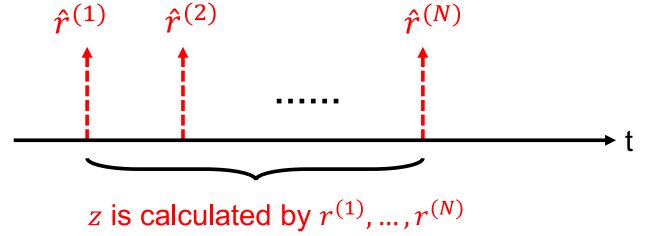


Fig. 14. Obtaining an estimated $s^{(i)}$ based on the ZCR value.

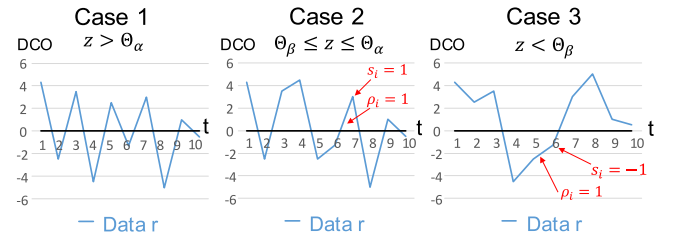


Fig. 15. Three cases for the ZCR value.

once we have determined $\hat{r}^{(i)}$. Since our target is $s^{(i)}$, a decision-making method is proposed that can acquire the final output $\hat{s}$ based on $\hat{\rho}$.

In order to obtain a suitable value for $\hat{s}$, a ZCR for the DCO data is introduced. As shown in Fig. 14, the ZCR for $r^{(1)}, r^{(2)}, \dots, r^{(N)}$ is denoted as z , and is defined as:

$$z = \frac{1}{N-1} \sum_{i=2}^N \frac{|\text{sgn}(r^{(i)}) - \text{sgn}(r^{(i-1)})|}{2}. \quad (14)$$

The value of z is obviously between 0 and 1. We can then determine the likelihood that the sign for the DCO changes from the ZCR value, which is classified by setting two thresholds, Θ_α and Θ_β , based on the ZCR value. There are a total of three cases, as shown in Fig. 15, including $z > \Theta_\alpha$, $\Theta_\beta \leq z \leq \Theta_\alpha$, and $z < \Theta_\beta$.

In Case 1, when the ZCR value is large enough, which implies that the sign of r acts in a more stable manner, the data is easier to learn and predict. In this situation, we let the estimated $\hat{s}^{(i)}$ be directly equal to $\text{sgn}(\hat{r}^{(i)})$. In Case 2, when the ZCR value is smaller than Θ_α , which implies that the actions of the data are not so stable, the result will become worse by letting $\hat{s}^{(i)} = \text{sgn}(\hat{r}^{(i)})$. However, suppose that the ZCR value is larger than Θ_β , $\rho^{(i)}$ will be equal to $s^{(i)}$ with a high probability. Conversely, in Case 3, when the ZCR value is smaller than

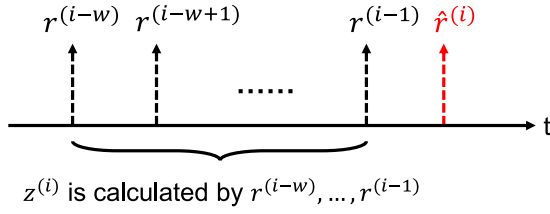


Fig. 16. Sliding ZCR.

Θ_β , $\rho^{(i)}$ will be equal to $s^{(i)}$ with a low probability. Based on these characteristics, the decision-making method will be applied according to

$$\hat{s}^{(i)} = \begin{cases} \text{sgn}(\hat{r}^{(i)}), & \text{if } z > \Theta_\alpha \\ +\hat{\rho}^{(i)}, & \text{if } \Theta_\beta \leq z \leq \Theta_\alpha \\ -\hat{\rho}^{(i)}, & \text{if } z < \Theta_\beta. \end{cases} \quad (15)$$

B. Sliding Window for the ZCR

We can observe from (14) that the calculation for the ZCR is actually based on the sequence $r^{(1)}, \dots, r^{(N)}$. However, in the inference stage, we can't obtain the true value of the sequence $r^{(1)}, \dots, r^{(N)}$. In order to obtain the ZCR, we introduce a sliding window into the ZCR method. As shown in Fig. 16, we use data from time $i - w$ to time $i - 1$, i.e., the window size is w , to calculate the ZCR $z^{(i)}$ and hence an estimated DCO sign in time i denoted as $\hat{s}^{(i)}$ can be obtained according to:

$$\hat{s}^{(i)} = \begin{cases} \text{sgn}(\hat{r}^{(i)}), & \text{if } z^{(i)} > \Theta_\alpha \\ +\hat{\rho}^{(i)}, & \text{if } \Theta_\beta \leq z^{(i)} \leq \Theta_\alpha \\ -\hat{\rho}^{(i)}, & \text{if } z^{(i)} < \Theta_\beta. \end{cases} \quad (16)$$

The decision-making method is based on the two thresholds, i.e., Θ_α and Θ_β , which are selected by evaluating the cross-validation samples. We will select those thresholds which best fit the cross-validation samples as the optimal thresholds to be used for the testing data. The choices of these thresholds are very important for the final output. Thus, we can't randomly choose cross-validation samples.

Referring to Fig. 17, suppose the sequence of the DCO values in the input for the inference is denoted as η_j^{input} and the sequence of the DCO values in the input for the training samples is denoted as η_i . Define the selection criteria $D_{j,i}$ as:

$$D_{j,i} = \frac{\|\eta_j^{input}, \eta_i\|}{S(\eta_j^{input}, \eta_i) + T(\eta_j^{input}, \eta_i)}, \quad (17)$$

where we use $\|\cdot\|$ to calculate the Euclidean distance between two vectors, and $S()$ and $T()$ are defined as follows:

$$S(\mathbf{x}, \mathbf{y}) = \sum_i \frac{|\text{sgn}(x_i) + \text{sgn}(y_i)|}{N}, \quad (18)$$

$$T(\mathbf{x}, \mathbf{y}) = \sum_i \frac{|\text{sgn}(x_i - x_{i-1}) + \text{sgn}(y_i - y_{i-1})|}{N - 1}, \quad (19)$$

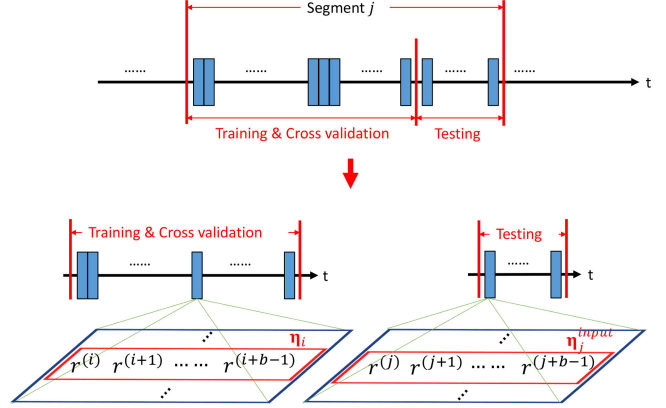


Fig. 17. Selection of the cross validation samples.

where N is the vector length. Each η_j^{input} can then identify a sample η_i , whose $D_{j,i}$ is the smallest, which will then act the cross-validation samples. The performance when using different criteria will be discussed in the next subsection.

C. Simulation Results

In the simulation section, we set the size of the sliding window in the ZCR model to be equal to 15. Based on the reason we described in Section III-B, we divide the original samples into 10 segments. The following results will be shown as the average performance for these 10 segments.

In addition to the prediction of the stock index value, such as the opening and closing prices, and the DCO value, we also evaluate the hit rate from the sign of the DCO value, i.e., the POS. Along with the POS, we also consider a second metric for the model known as the Prediction of the Direction (POD) so as to evaluate the accuracy of $\hat{\rho}$, where POD is calculated from:

$$\text{POD} = \frac{\sum_{i=2}^N |\rho^{(i)} + \hat{\rho}^{(i)}|/2}{N - 1}. \quad (20)$$

The following explanation illustrates the defects when directly taking the sign of the estimated r as an estimate of s . Table VII presents the related results when the price range model is used where no decision making is conducted, i.e., $\hat{s}^{(i)} = \text{sgn}(\hat{r}^{(i)})$. From the table, we can see that the value for the POS is only about 50%, i.e., basically a random guess. Obviously, it is not feasible to estimate $s^{(i)}$ by using the sign of $\hat{r}^{(i)}$. For $\hat{\rho}$, on the other hand, the trend for the stock can be captured by the model very well, since an accuracy of almost 80% is achieved by the POD. The value of s can be forecast more accurately when enhanced by ρ .

Fig. 18 presents a comparison when using different criteria to select cross-validation samples. Fig. 18(a) illustrates the data selected using the criteria defined in (17). Fig. 18(b) is the data selected using the criteria defined as $\|\eta_j^{input}, \eta_i\|$. Although the data illustrated in Fig. 18(b) has the smallest Euclidean distance, η_i can't simulate η_j^{input} well. On the other hand, (17) not only requires a small Euclidean distance, but also a small Hamming distance for the sign and a small Hamming distance for the trend.

TABLE VI
COMPARISONS OF THE ACCURACY OF THE TPR AND THE TNR FOR THE DCO FROM THE S&P500 AND DOW JONES DATA USING THE SVM,
THE LSTM AND OUR PROPOSED METHOD

	Testing Days (N)	SVM			LSTM			M		
		POS	TPR	TNR	POS	TPR	TNR	POS	TPR	TNR
SP	$N = 15$	0.447	0.245	0.756	0.453	0.455	0.558	0.592	0.625	0.594
	$N = 20$	0.505	0.218	0.808	0.523	0.343	0.632	0.594	0.602	0.614
DJ	$N = 15$	0.433	0.232	0.783	0.445	0.211	0.892	0.585	0.497	0.711
	$N = 20$	0.483	0.120	0.914	0.485	0.225	0.886	0.573	0.426	0.772

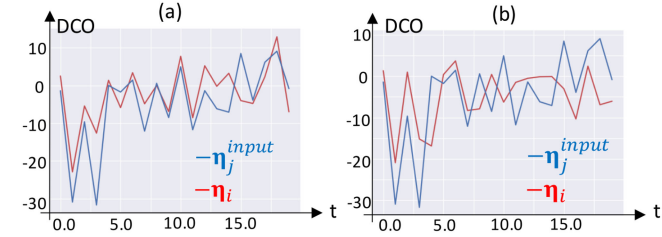


Fig. 18. A comparison of different criteria used to select cross validation samples.

		Actual Class	
		1	-1
Predicted Class	1	True Positive (TP)	False Positive (FP)
	-1	False Negative (FN)	True Negative (TN)

Fig. 19. Confusion matrix.

Thus, we will use (17) as our criteria for selecting the cross-validation samples.

We now use the confusion matrix shown in Fig. 19 to analyze our model.

The confusion matrix is a table that is often used to describe the classification model. A positive value indicates the stock is rising, while a negative value indicates the stock is falling. In stock prediction, we are concerned with both rising and falling cases in order to decide whether to buy or sell according to the predicted direction. Sometimes, the prediction of the sign (i.e., $\frac{TP+TN}{TP+TN+FP+FN}$) is not a reliable metric, since the data set is unbalanced in different classes. Two metrics, including the true positive rate (TPR) and the true negative rate (TNR), in addition to the accuracy, will be used to evaluate our model. TPR and TNR can be computed according to

$$TPR = \frac{TP}{TP + FN}, \quad TNR = \frac{TN}{TN + FP}. \quad (21)$$

Table VI provides a comparison of the POS, the TPR and the TNR between the SVM, the LSTM, and the proposed method,

TABLE VII
THE POS AND THE POD FOR THE S&P500

	POS ₂₀	POD ₂₀	POS ₁₅	POD ₁₅
Average	0.455	0.815	0.5	0.787

where we can see that the proposed method achieves the highest POS value. In contrast, however, both the SVM and the LSTM are only able to achieve an accuracy of 50% or less. This indicates that the use of ZCR can indeed improve the POS performance.

When using our proposed model for the S&P500, under the condition where the actual value is either 1 or -1, both the TPR and the TNR can achieve accuracy of 60% or more, which means that it has a good ability for prediction in both directions. For the Dow Jones, under the condition when the actual value is 1, the performance of the TPR for all the models is very poor. In contrast, under the condition where the actual value is -1, the TNR can achieve very high accuracy. However, our proposed approach achieves a better performance for the TPR than the others. We also achieve a better performance for the POS.

The FNR and, respectively, the FPR, are the predicted error rate, which can be calculated using $FNR = 1 - TPR$ and $FPR = 1 - TNR$. When the TPR and the TNR are higher, the FNR and the FPR are lower, which means that there is a lower predicted error rate. As they tend to select the same class for all, both the SVM and the LSTM produce this type of result, i.e., Low for the TPR and High for the TNR.

In addition to the hit rate, after obtaining a better estimated sign value, i.e., $\hat{s}^{(i)}$, the estimated DCO value can be re-obtained using $|\hat{r}^{(i)}| \times \hat{s}^{(i)}$. Suppose that the estimated DCO value enhanced by $\hat{s}^{(i)}$ is denoted as $\hat{r}_e^{(i)}$. Table VIII shows the regression errors compared with different methods. Model M_p is the model we proposed in Section III, and the regression errors are computed with respect to $\hat{r}^{(i)}$. Model M_{DCO} is the model we proposed in Section IV, and the regression errors are computed with respect to $\hat{r}_e^{(i)}$. By observing from the table, we can see that most of the results achieve the lowest error, even when compared to the other methods, such as SVM and LSTM.

In addition, we also compare our model to those described in previous research. Table IX shows a comparison of our model with those proposed in previous research. The authors of [34] applied both ANN and SVM to predict the SET50 index,

TABLE VIII
REGRESSION ERRORS ENHANCED USING $\hat{s}$ FOR THE DCO VALUE FROM THE S&P500 AND DOW JONES DATA FOR DIFFERENT TESTING DAYS

	Testing Days (N)	M_p		M_{DCO}		SVM		LSTM	
		RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
SP	$N = 15$	15.787	12.778	12.933	10.44	13.3	10.8	15.7	12.7
	$N = 20$	15.385	12.51	13.265	10.76	13.2	10.5	13.1	10.6
DJ	$N = 15$	122.23	100.64	106.47	84.23	112.3	88.7	122.0	100.6
	$N = 20$	120.23	96.255	112.3	88.83	112.2	88.6	113.6	88.6

TABLE IX
COMPARISON OF THE ACCURACY FOR OUR MODEL AND OTHER RESEARCH

Methods	M_{DCO}	ANN [34]	LSTM+GCN [21]	NBAd [26]	CNN [20]
Data	S&P500	SET50	CSI300	S&P500	S&P500
ACC	59.4%	56.3%	57.98%	58.1%	56.14%

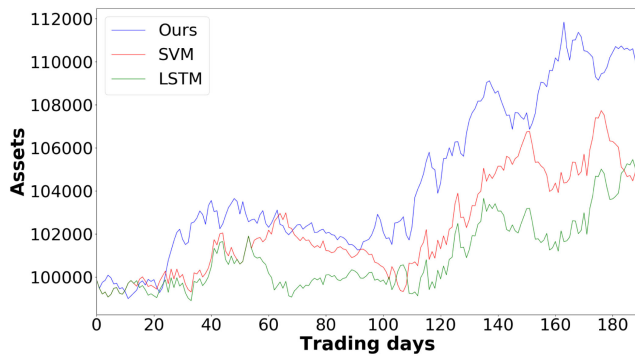


Fig. 20. Comparison of the trading performance achieved by the proposed method, the SVM, and the LSTM.

while those of [21] analysed the relationship between a group of companies and used LSTM to predict the stock price. The authors of [26] proposed a method of combining numerical data and financial news to predict the stock price. A time series reconstruction combined with a CNN was proposed in [20] to predict the stock trends. It can be seen from Table IX that our model achieves a comparable performance to these other models.

In order to test our model in practice, we applied our simulation results to perform an historical test of the trading performance, and applied a trading strategy to perform historical testing based on the S&P500. We concatenated the testing day in each segment to construct the data. The trading strategy was that if the prediction for the stock price was that it would rise the next day, we bought it at its opening price, and sold it at its closing price. If the prediction was that the price would fall the next day, we do the opposite. Fig. 20 shows the trading performance compared to the SVM and the LSTM. Though our model does not achieve a 100% accuracy, the result indicates that our method is still profitable.

V. CONCLUSION

The DCO provides crucial insights in the market, including making profits and taking long or short positions. Besides the values of DCO, our focus is typically on the sign of the DCO. In an attempt to obtain a better estimate for the sign for the DCO, our method focused more on the characteristics of the stock data, such as the ZCR. In comparison with prior arts, our method obtains a better hit rate for the DCO signs, while also achieving a similar performance in regression error for the DCO values.

In the future, we may also apply Natural Language Processing (NLP) to news and financial reports to obtain new features or use some statistical methods to enhance our model. In order to apply our model to actual practice, some trading strategies will also be devised.

REFERENCES

- [1] I. Aldridge, *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. Hoboken, NJ, USA: Wiley, 2013.
- [2] M.-E. Wu and W.-H. Chung, "A novel approach of option portfolio construction using the kelly criterion," *IEEE Access*, vol. 6, pp. 53 044–53 052, 2018.
- [3] M.-E. Wu, C.-H. Wang, and W.-H. Chung, "Using trading mechanisms to investigate large futures data and their implications to market trends," *Soft Comput.*, vol. 21, pp. 2821–2834, May 2016.
- [4] M.-E. Wu, C.-H. Wang, W.-H. Chung, R. Tso, and I.-H. Yang, "An empirical comparison between kelly criterion and vince's optimal F," in *Proc. IEEE Int. Conf. Smart City/SocialCom/SustainCom (SmartCity)*, Dec. 2015, pp. 806–810.
- [5] M.-E. Wu and W.-H. Chung, "Empirical evaluations on momentum effects of Taiwan index futures via stop-loss and stop-profit mechanisms," *Int. J. Inf. Technol. Decis. Making*, vol. 18, pp. 1–20, Nov. 2018.
- [6] S. M. Chen and C. D. Chen, "TAIEX forecasting based on fuzzy time series and fuzzy variation groups," *IEEE Trans. Fuzzy Syst.*, vol. 19, no. 1, pp. 1–12, Feb. 2011.
- [7] P. Chang and C. Fan, "A hybrid system integrating a wavelet and TSK fuzzy rules for stock price forecasting," *IEEE Trans. Syst., Man, Cybern., Part C (Appl. Rev.)*, vol. 38, no. 6, pp. 802–815, Nov. 2008.
- [8] E. Hadavandi; H. Shavandi; A. Ghanbari, "A genetic fuzzy expert system for stock price forecasting," in *Proc. IEEE Int. Conf. Fuzzy Syst. Knowl. Discovery (FSKD)*, vol. 1, Aug. 2010, pp. 41–44.

- [9] A. A. Ariyo, A. O. Adewumi, and C. K. Ayo, "Stock price prediction using the ARIMA model," in *Proc. IEEE Int. Conf. Comput. Modelling Simul.*, Mar. 2014, pp. 106–112.
- [10] S. M. Idrees, M. A. Alam, and P. Agarwal, "A prediction approach for stock market volatility based on time series data," *IEEE Access*, vol. 7, pp. 17287–17298, 2019.
- [11] S. Wichaidit, S. Kittitornkun, "Predicting SET50 stock prices using CARIMA (cross correlation ARIMA)," in *Proc. IEEE Int. Comput. Sci. Eng. Conf.*, Nov. 2015, pp. 1–4.
- [12] M. J. Kane, N. Price, M. Scotch, and P. Rabinowitz, "Comparison of ARIMA and random forest time series models for prediction of Avian influenza H5N1 outbreaks," *BMC Bioinf.*, vol. 15, no. 1, p. 276, 2014.
- [13] L. J. Cao and F. E. H. Tay, "Support vector machine with adaptive parameters in financial time series forecasting," *IEEE Trans. Neural Netw.*, vol. 14, no. 6, pp. 1506–1518, Nov. 2003.
- [14] Y. Lin, H. Guo, and J. Hu, "An svm-based approach for stock market trend prediction," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, Aug. 2013, pp. 1–7.
- [15] G. C. Cawley and N. L. C. Talbot, "Over-fitting in model selection and subsequent selection bias in performance evaluation," *J. Mach. Learn. Res.*, vol. 11, pp. 2079–2107, Jul. 2010.
- [16] Y. B. Wijaya, S. Kom, and T. A. Napitupulu, "Stock price prediction: Comparison of ARIMA and artificial neural network methods - an Indonesia stock's case," in *Proc. IEEE Int. Conf. Adv. Comput., Control, Telecommun. Technol.*, Dec. 2010, pp. 176–179.
- [17] A. Ntakaris, G. Mirone, J. Kannianen, M. Gabbouj, and A. Iosifidis, "Feature engineering for mid-price prediction with deep learning," in *IEEE Access*, vol. 7, pp. 82390–82412, 2019.
- [18] K. S. Zarkias, N. Passalis, A. Tsantekidis, and A. Tefas, "Deep reinforcement learning for financial trading using price trailing," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, May 2019, pp. 3067–3071.
- [19] M. Yu and J. Wu, "Ceam: A novel approach using cycle embeddings with attention mechanism for stock price prediction," in *Proc. IEEE Int. Conf. Big Data Smart Comput. (BigComp)*, Feb. 2019, pp. 1–4.
- [20] M. Wen, P. Li, L. Zhang, and Y. Chen, "Stock market trend prediction using high-order information of time series," *IEEE Access*, vol. 7, pp. 28299–28308, 2019.
- [21] Y. Chen, Z. Wei, and X. Huang, "Incorporating corporation relationship via graph convolutional neural networks for stock price prediction," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manag.*, ser. CIKM '18. New York, NY, USA: ACM, 2018, pp. 1655–1658.
- [22] A. Graves, A. Mohamed, and G. Hinton, "Speech recognition with deep recurrent neural networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, May 2013, pp. 6645–6649.
- [23] K. Dutta and K. K. Sarma, "Multiple feature extraction for RNN-based assamese speech recognition for speech to text conversion application," in *Proc. IEEE Int. Conf. Commun., Devices Intell. Syst.*, Kolkata, India, Dec. 2012, pp. 600–603.
- [24] T. Gao, Y. Chai, and Y. Liu, "Applying long short term memory neural networks for predicting stock closing price," in *Proc. IEEE Int. Conf. Softw. Eng. Service Sci.*, Beijing, China, Apr. 2017, pp. 575–578.
- [25] L. Zhang, C. Aggarwal, and G.-J. Qi, "Stock price prediction via discovering multi-frequency trading patterns," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, ser. KDD '17. New York, NY, USA: ACM, 2017, pp. 2141–2149.
- [26] G. Liu and X. Wang, "A numerical-based attention method for stock market prediction with dual information," *IEEE Access*, vol. 7, pp. 7357–7367, 2019.
- [27] L. Shi, Z. Teng, L. Wang, Y. Zhang, and A. Binder, "DeepClue: Visual interpretation of text-based deep stock prediction," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 6, pp. 1094–1108, Jun. 2019.
- [28] X. Li, H. Xie, R. Y. K. Lau, T. Wong, and F. Wang, "Stock prediction via sentimental transfer learning," *IEEE Access*, vol. 6, pp. 73110–73118, 2018.
- [29] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. USA: MIT Press: Cambridge, MA, 2016.
- [30] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [31] T. Robert, "Regression shrinkage and selection via the lasso," *Proc. IEEE*, vol. 58, no. 1, pp. 267–288, 1996.
- [32] B. Léon, *Large-Scale Machine Learning With Stochastic Gradient Descent*. Berlin, Germany: Springer, 2010, pp. 177–186.
- [33] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis.*, Santiago, Chile, Dec. 2015, pp. 1026–1034.
- [34] M. Inthachot, V. Boonjing, and S. Intakosum, "Predicting SET50 index trend using artificial neural network and support vector machine," in *Current Approaches in Applied Artificial Intelligence*, pp. 404–414, 2015.
- [35] Y. Lin, Y. Ueng, W. Chung, and T. Huang, "Stock price range forecast via a recurrent neural network based on the zero-crossing rate approach," in *Proc. IEEE Conf. Comput. Intell. Financial Eng. Econ. (CIFER)*, 2019, pp. 1–9.



Yu-Fei Lin received the B.S. degree from the Department of Electronic Engineering, National Chung Cheng University, Chiayi, Taiwan, in 2016. He is currently working toward the master's degree with the Institute of Communications Engineering, National Tsing Hua University, Hsinchu, Taiwan. His research interests include channel coding, machine learning.



Tzu-Ming Huang received the B.S. degree from the Department of Communication Engineering, National Central University, Taoyuan, Taiwan, in 2017. He is currently working toward the Ph.D. degree with the Institute of Communications Engineering, National Tsing Hua University, Hsinchu, Taiwan. His research interests include channel coding, machine learning, and FinTech.



Wei-Ho Chung received the B.Sc. and M.Sc. degrees in electrical engineering from National Taiwan University, Taipei City, Taiwan, in 2000 and 2002, respectively, and the Ph.D. degree from the Electrical Engineering Department, University of California, Los Angeles, CA, USA, in 2009. He received a Taiwan Merit Scholarship from 2005 to 2009, and the Best Paper Award at IEEE WCNC 2012, the Ta-You Wu Memorial Award 2016, and has authored or coauthored more than 50 refereed journal articles and more than 50 refereed conference papers. He has been a Tenure-Track Assistant Professor since January 2010, with the Research Center for Information Technology Innovation, Academia Sinica, and was promoted to an Associate Professor in January 2014. At Academia Sinica, he supervised approximately 4–6 full-time postdocs and research assistants, and jointly advises multiple Ph.D. students with faculty members in various universities. Since February 2018, he has held the position of a Full Professor and leads the Wireless Communications Lab, Electrical Engineering Department, National Tsing Hua University, Taiwan. His research interests include communications, signal processing, and networks.



Yeong-Luh Ueng (Senior Member, IEEE) received the Ph.D. degree in communication engineering from the National Taiwan University, Taipei, Taiwan, in 2001. From 2001 to 2005, he was with a Private Communication Technology Company, where he focused on the design and development of various wireless chips. Since December 2005, he has been a member of the faculty with the National Tsing Hua University, Hsinchu, Taiwan, where he is currently a Full Professor with the Department of Electrical Engineering and the Institute of Communications Engineering. In 2016, he was the recipient of the Wu Ta-You Memorial Award from the Ministry of Science and Technology (MOST). In 2018, he was the recipient of the Outstanding Electrical Engineering Professor from the Chinese Electrical Engineering Association, and the Outstanding Research Award from the Ministry of Science and Technology (MOST), Taiwan. His research interests include coding theory, wireless communications, and communication ICs.